

Reading Assignment

1. What are the advantages of Polymorphism?

Đa hình (Polymorphism) mang lại những ưu điểm lớn trong lập trình hướng đối tượng, bao gồm:

- **Tính mở rộng (Extensibility):** Hệ thống rất dễ dàng được mở rộng. Bạn có thể thêm các lớp con mới vào hệ thống mà không cần phải thay đổi hay biên dịch lại các đoạn mã nguồn cũ đang xử lý dựa trên lớp cha hoặc giao diện (interface).
- **Tính tái sử dụng và ngắn gọn (Reusability & Simplicity):** Cho phép viết một đoạn mã duy nhất để xử lý nhiều loại đối tượng khác nhau. Ví dụ: sử dụng một vòng lặp for để gọi phương thức toString() hoặc play() trên một danh sách (ArrayList) chứa chung các đối tượng lớp cha, thay vì phải viết các vòng lặp hoặc khối lệnh if-else riêng biệt cho từng lớp con.
- **Tính linh hoạt (Flexibility):** Cho phép thay đổi hành vi của hệ thống tại thời điểm chạy (runtime) dựa trên đối tượng thực tế được truyền vào.

2. How is Inheritance useful to achieve Polymorphism in Java?

Kế thừa (Inheritance) chính là nền tảng thiết yếu để Đa hình có thể hoạt động trong Java:

- **Tạo ra mối quan hệ "IS-A":** Kế thừa giúp định nghĩa các lớp con thuộc về một kiểu lớp cha chung (Ví dụ: Book là một loại Media). Nhờ đó, Java cho phép sử dụng một biến tham chiếu kiểu lớp cha để trỏ tới các đối tượng của lớp con (Upcasting).
- **Cơ chế Ghi đè (Overriding):** Kế thừa cung cấp khả năng cho phép lớp con định nghĩa lại (ghi đè) các phương thức đã có ở lớp cha.
- **Ràng buộc muộn (Late Binding / Dynamic Dispatch):** Khi gọi một phương thức thông qua biến tham chiếu của lớp cha, nhờ có cấu trúc phân cấp từ kế thừa, Java có thể tự động kiểm tra xem đối tượng thực sự đang được trỏ tới ở bộ nhớ là lớp con nào để thực thi đúng phương thức đã được ghi đè của lớp con đó tại thời điểm chương trình chạy (runtime).

3. What are the differences between Polymorphism and Inheritance in Java?

- **Về mục đích cốt lõi:**
 - **Kế thừa (Inheritance)** thiên về **cấu trúc**. Mục đích chính là tái sử dụng mã nguồn (thuộc tính, phương thức) từ một lớp đã có để tạo ra lớp mới, giúp giảm thiểu việc lặp code và tạo ra hệ thống phân cấp logic.
 - **Đa hình (Polymorphism)** thiên về **hành vi**. Mục đích chính là cho phép một thông điệp (một lời gọi phương thức) có thể được phản hồi theo nhiều cách khác nhau tùy thuộc vào đối tượng thực tế đang nhận thông điệp đó.
- **Về góc nhìn thiết kế:**
 - Kế thừa định nghĩa việc "Lớp này là **một** (IS-A) cái gì?".
 - Đa hình định nghĩa việc "Đối tượng này **thực hiện** hành động đó như thế nào?".

- **Về thời điểm thể hiện:**

- Kế thừa được thiết lập chặt chẽ và thể hiện cấu trúc tĩnh ngay từ **thời điểm biên dịch (Compile-time)**.
- Đa hình (cụ thể là đa hình động thông qua overriding) thể hiện sức mạnh linh hoạt của nó tại **thời điểm chạy (Runtime)**.